

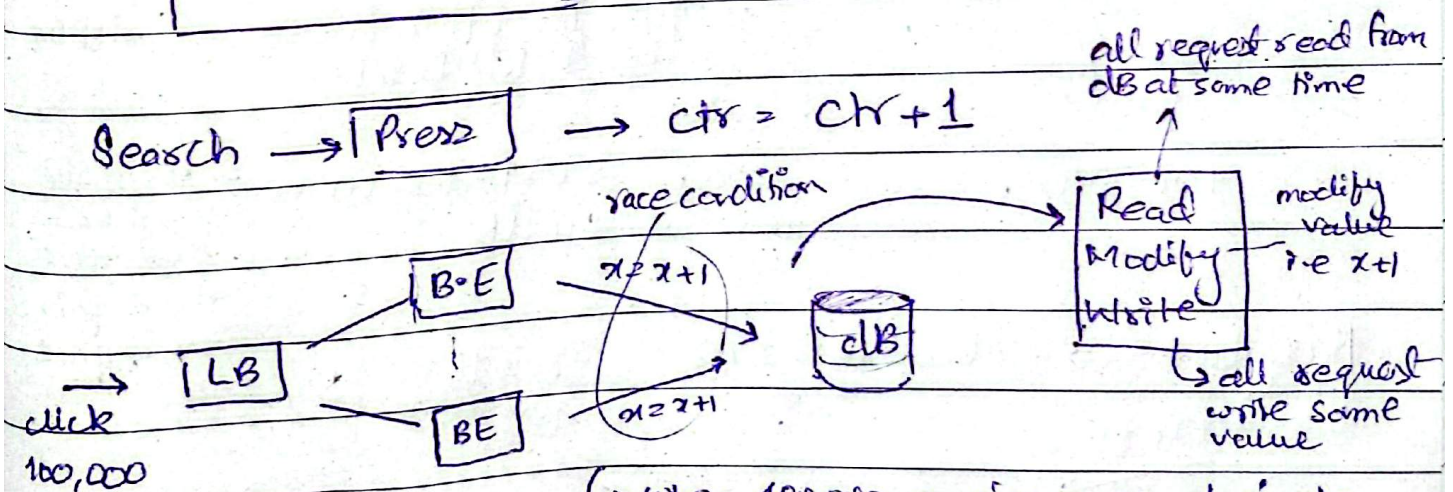
Date _____

Day _____

Introduction To Write Intensive System

→ Need to design an analytics system for Booking.com
which tells how many users have pressed the search btn
↳ We have 100,000 QPS - 100,000 clicks/sec

Analytics Table	
id	
Search_btn_ctr	
⋮	



↳ when 100,000 queries comes at single point of time they will all get same value and we won't be able to get correct analytics

Page No. _____

Date _____

Day _____

• Business Requirement?

↳ Real-Time System $\Rightarrow$ expensive to build

↳ Results need to be conveyed immediately

↳ ex: Stock Market, Gaming Leaderboard

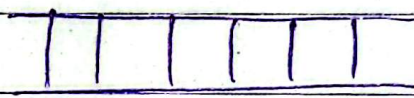
↳ 100,000 QPS $\rightarrow$ 2sec $\rightarrow$ 200,000 clicks
should be shown on dashboard

↳ Non-Real-Time System

↳ Delay is affordable

the throughput of this queue should be 100,000 QPS

counter
update



← batch →

→ B.E

100



↳ request for ctr increment will be processed in batch

$t=0$

100,000

$t=1$

200,000

150,000 $\rightarrow$ in dB

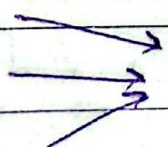
$\rightarrow$ expected ctr val

these two will always be diff the gap incr. over time

Kafka $\rightarrow$ million msgs/s

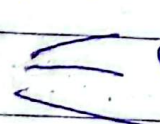
ctr + 1

Producers



50 MB/s

can directly perform aggregation

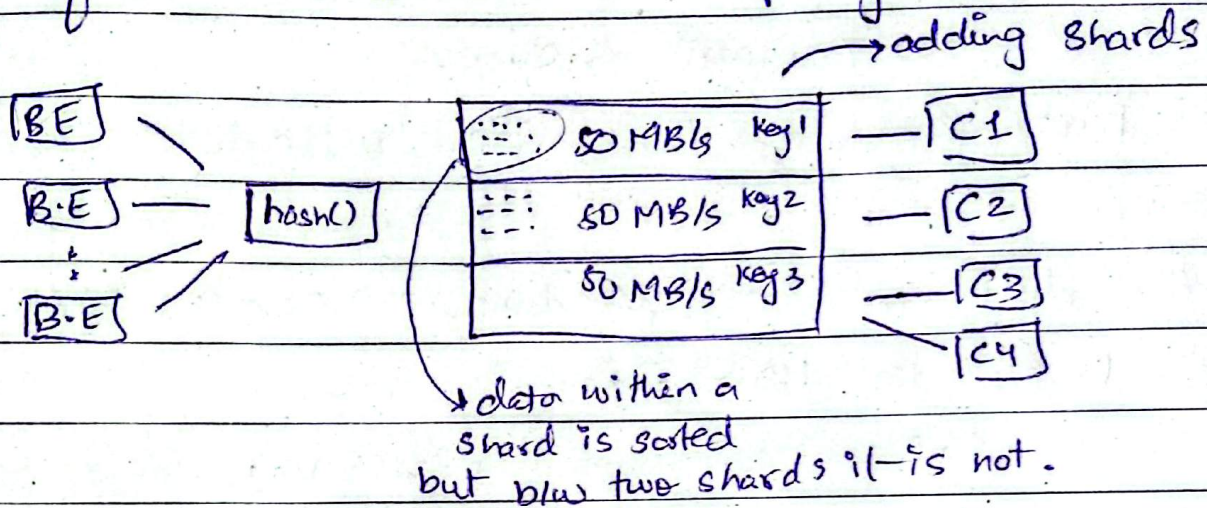


Consumers

Date _____

Day _____

if we need to increase the capacity,



↳ The Process Of Writing in a dB

(1) Client (B-E) → sends an INSERT/UPDATE Query

(2) Changes saved in buffer only. → can be cache pool

↳ Buffer → in-memory data structure

(3) B-Trees are then updated

↳ Rebalancing, splitting, merging

↳ Pointers gets updated

(4) A new entry in a log file create.

↳ An Append only log file called "Write Ahead Log"

↳ It is a change log in which every change to dB is written

↳ Provides Durability to dB

Page No. _____

Date _____ Server crash hoga, then WAL will be replayed
Day _____

⑤ WAL → flush to disk

↳ COMMIT ⇒ success

⑥ Dirty pages are then written to the disk

• Why DB cannot be 100% durable even after WAL?

↳ due to HDD/SDD

↳ Every HDD has 5% error rate/year

↳ Even storing backups doesnot solve

↳ what if a faulty primary DB's backup is made?

↳ what if a error prone backup is used to instantiate a primary DB?

• Transactions

↳ Either all the operations will succeed or none.

↳ Atomicity.

Date _____

Day _____

With Transaction	Without Transaction
Begin	
UPDATE USERS	UPDATE Users
SET bal = 100	SET bal = 100
WHERE id = 1	WHERE id = 1 — id = 100
id = 100	
End	↳ Each query will write its own log.
↳ the WAL will be update after the completion of whole transaction	↳ 100 writes on WAL
↳ only single write on WAL	

• Consistency

- ↳ To maintain the consistency of DB is the responsibility of the Backend.
- ↳ DB provides us feature like CONSTRAINTS, FK, etc to implement consistency.

Date _____

Day _____

• Isolation

↳ Multiple transactions should not affect the output of each other.

↳ To ensure 100% isolation, the perfect and only soln is to run all transactions sequentially one by one.

↳ Degrades performance too much.

↳ All Relational DB's provide "Weak Isolation" Guarantee.

• Concurrency

↳ Ability to Allow multiple users or applications to access & modify shared data at same time.

↳ Problems due to Concurrency

(1) Dirty Read → Reading an uncommitted value

T ₁	T ₂
Read(x)	
Write(x)	
⋮	Read(x)
⋮	Commit()
Rollback()	

Page No. _____

Date _____

Day _____

② Read Skew (Unrepeated Read)

↳ Reading a value two times and getting diff value each time

	T_1	T_2	
$x = 10$	Read(x)		
.		Read(x)	$x = 10$
$x++$	Write(x)		
		Read(x)	$x = 11$

} two diff values of x

③ Phantom Read

→ not able to read a value and time bcz of change in the structure of db.

	T_1	T_2	
$x = 10$	Read(x)		
		Read(x)	$x = 10$
	Delete(x)		
		Read(x)	

④ Lost Update

T_1 would think that 10 would have been committed

$x = 10$

T_1	T_2
Read(x)	
Write(x) → $x = 11$	
	Write(x) → $x = 20$
Commit	Commit

Page No. _____

Date _____

Day _____

⑤ Dirty Write

↳ Transaction overwrites an uncommitted data

T ₁	T ₂
Read(x)	
Write(x)	
	Read(x)
	Write(x)
Rollback	

→ overwrites T₁'s uncommitted value.

⑥ Write Skew

↳ Two simultaneous transactions read the same data, but then update diff objects in a way that violates business rule.

↳ Solutions

- ① Read Comitted → Only read data from a committed transaction
- ② Snapshot Isolation → Used to prevent repeated read prob
↳ Each transaction takes its own snapshot of data and updates on that.

Page No. _____

- Predicate Lock

↳ lock is applied on a condition (criteria) instead of a specific row

↳ ex: Trans A runs `SELECT * FROM users WHERE dep = "Sales"`. The lock is placed on condition "`dep = Sales`".

Date _____

Day _____

③ Multi Version Concurrency Control

↳ Let multiple users read and write data concurrently without blocking each other

↳ Instead of overwriting old records during an update the DB creates a new version of the data, allowing readers to safely access older snapshots while writers modify new ones.

- Locking Mechanism

- Shared lock / Read-lock (r)

↳ If a transaction only needs to read then use this read-lock (r).

↳ Multiple trans can acquire read lock at same time

- Exclusive / Write-lock (w)

↳ If a transaction wants to read & write both then it will acquire write-lock (w).

↳ No two trans can have this lock at same time. Others need to wait for this lock to be free.

↳ Another trans can also not acquire read-lock (r) until write-lock (w) is free.

Page No. _____

Date _____

Day _____

- Deadlock can occur if transactions are dependent on each other.

- Room Booking Usecase. → Prevent double booking

① Exclusive Lock

↳ Remaining rooms column per lock lagadenge so only one transaction can update value.

② Constraint

↳ Apply db constraint on remaining_rooms ≥ 0 .

↳ So transaction will get rollback

→ Transaction Commit

→ 2-Phase Committing

↳ Used to handle transactions in a "Distributed System".

↳ A "coordinator" oversees multiple participants and ensures that either all participants agree to commit transaction, or if any one fails to commit, all participants roll back.

Page No. _____

Date _____

Day _____

- ↳ If coordinator is unavailable, the sys goes in a deadlock stage
- ↳ Used where consistency is more critical than availability.

→ Username Lookup Usecase.

↳ To efficiently check whether the username is valid or not. (checking uniqueness)

↳ If we query on DB it is expensive

↳ For this purpose Bloom Filters are used

↳ Based on probability

↳ If it says that no username exists mean 100% correct

↳ But if it says that username "can" exist then we query the DB.

not sure